Checking Every Proposal: A Fail-Closed Flood Decision-Support Chain from Forecast to Dispatch

Nandhakumar T
Department of Information Technology
Sona College of Technology
Salem, Tamil Nadu, India
nandhakumartm93@gmail.com

Balakarthikeyan B S
Department of Artificial Intelligence and Data Science
Sona College of Technology
Salem, Tamil Nadu, India
balasan2626@gmail.com

Gowthaman J
Department of Artificial Intelligence and Data Science
Sona College of Technology
Salem, Tamil Nadu, India
gowthamanjanadhran@gmail.com

Abhishek S
Department of Artificial Intelligence and Data Science
Sona College of Technology
Salem, Tamil Nadu, India
abisheksengodan18@gmail.com

***Abstract*—Flood decision-support software combines components that can be wrong without warning: a learned forecaster, sampling optimizers such as the Quantum Approximate Optimization Algorithm (QAOA), and large language models (LLMs) that narrate the situation. In our prototype each component only proposes, and a deterministic check at its boundary decides what reaches the officer; we measure every check. A gradient-boosted forecaster scored on CAMELS-IND gauges unseen in space and time reached a balanced accuracy of 0.898 for three-day flood exceedance, against 0.873 for the dataset's LSTM simulation; a random row split gave 0.915 on its own test set. A constraint-preserving mixer raised QAOA's probability of the optimal plan from at most 0.048 to 0.188. One angle set chosen without optima raised the chance that 32 verified samples contain the optimum on fresh city instances from 80.1% to 96.8%, but sets chosen on the twin by energy or CVaR fell below per-instance training. Unchecked, the most frequent sample was invalid in 29.1% of re-planning events for QAOA and 20.6% for simulated annealing; an exact verifier with a mixed-integer certificate dispatched none, and calling the solver first with a 1 s deadline gave an optimal plan in 82.5% of solvable city calls. A composed check found that dispatch bypassed the verified policy in 2,807 of 32,032 cases, and 36.3% of 300 live LLM replies stated a number absent from the state; a slot-typed guard flagged 93.6% of them. QAOA was simulated classically; we claim no quantum advantage.**

***Index Terms*—flood forecasting, runtime assurance, QAOA, large language models, fail-safe design, evacuation planning**

I. Introduction

During a flood, an emergency operations centre turns a forecast into a decision, the decision into a shelter plan and the plan into radio messages. Each step is now offered to a new kind of component: data-driven forecasters trained on many catchments [1], [2], quantum and quantum-inspired optimizers for assignment problems [3]–[5], and LLMs that plan, call tools and speak [6]. They fail in different ways. A forecaster's reported skill depends on how its test data were split [7], [8]. A sampler returns candidates, not proven answers, and the penalty encodings it needs for capacity limits are exact only in special cases [9]. An LLM can state numbers its input does not contain [10].

Safety engineering has a standard answer for unreliable parts: keep them, but put a simple checked element between them and the action [11], [12], as shields do for learned policies [13], and choose the level of automation per function [14]. For a whole flood system three questions remain open. Does each check hold on the path to action as it is actually built? What does each check cost in false alarms and lost quality? And what should the system do when a check cannot decide?

We study these questions on an open flood decision-support prototype with a digital twin of the Delhi Yamuna corridor, OpenStreetMap road networks of six Indian cities and a forecasting benchmark on CAMELS-IND [15]. Our contributions are:

1) A fail-closed chain in which the forecaster, the re-planner and the narrator can only propose, and a deterministic check at each boundary decides what reaches the officer (Fig. 1, Table I).
2) A forecast evaluation on gauges held out in space and time, with the effect of the evaluation protocol measured.
3) Shelter re-planning with QAOA under a constraint-preserving mixer and transferable angles, run inside a runtime-assurance loop with exact verification and a certificate for escalation.
4) Measurements on the deployed agent code: a composed check from policy to dispatch, and the grounding of live LLM replies under three guards.

Separate papers study each layer in depth; this one joins them and reuses their measurements.

II. Related Work

Forecasting. Long short-term memory (LSTM) networks trained on large-sample data simulate streamflow well, even in ungauged basins [1], [2]. Random row splits let days of

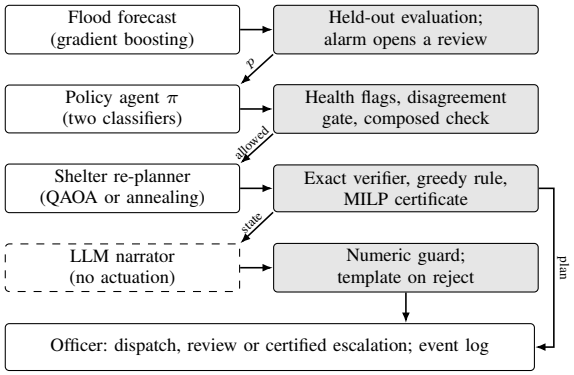


Fig. 1. The fail-closed chain. Left: components that can be wrong and only propose. Shaded: the deterministic check at each boundary, which can hold, downgrade or reject but never act. The LLM (dashed) narrates the logged state and has no actuation rights.

one flood fall on both sides of the split, which inflates scores [7], [8]; we score on gauges and years that played no part in training.

Quantum optimization. QAOA alternates a cost and a mixing unitary [16]; constraint-preserving mixers keep the state among feasible plans [17], [18], and rescaled instances can share one set of angles [19]. These studies use equality constraints or unconstrained MaxCut, whereas shelter capacity is an inequality. Quantum annealing has been applied to disaster resource allocation with a penalty encoding and no check of the returned samples [5].

Runtime assurance and LLM guards. The Simplex architecture lets a verified controller replace a complex one [11], and SOTER brings this to robotics software [12]. For LLMs, programmable rails [20] and runtime enforcement for agents [21] restrict what a model may do, and a second LLM can act as a judge [22]. Safety, however, is a property of the whole control structure [23]. To our knowledge no flood system has been evaluated with checks on its forecaster, its optimizer and its narrator together, on the code that runs.

III. System

The prototype runs one pipeline per tick, either on live weather and river data or on a digital twin of the corridor. Eleven agents share one state object and write every event to a log, so the officer sees what each agent did and why. Fig. 1 shows the chain. The rule is the same at every layer: a component that can be wrong produces a proposal, and a deterministic check decides whether it passes, is replaced by a safe default or goes to a human. Checks can hold, downgrade or reject; none can start an action. Table I summarises each layer and its main measured result, which Sections IV–VII report in turn.

IV. Forecast Layer

Task and split. For gauge g on day t the target is

$$y_{g,t} = \mathbb{1}\Big[\max_{h\in\{1,2,3\}} Q_{g,t+h} \;\geq\; q_g^{95}\Big], \tag{1}$$

TABLE I
Layers of the chain, their checks and the main measured result

Layer	Proposes	Check	Result
Forecast (§IV)	flood within 3 days	unseen gauges and years; review on alarm	BA 0.898, precision 0.328
Re-planner (§V, §VI)	shelter plan (samples)	exact verifier, greedy rule, MILP	0 invalid plans dispatched; 360/360 cases settled by the MILP
Policy (§VII)	dispatch	health flags, gate, composed check	bypasses 2,807 → 0
Narrator (§VII)	radio text	typed numeric guard	recall 93.6%, false rejection 10.5%

TABLE II
Three-day flood exceedance on 48 unseen gauges, 2015–2020

Model	BA (95% CI)	PR-AUC	Recall	Precision
XGBoost (selected)	0.898 (0.886–0.910)	0.737	0.902	0.328
XGBoost, no flow	0.853 (0.820–0.880)	0.494	0.865	0.239
LSTM simulation	0.873 (0.829–0.911)	0.589	0.826	0.376
Climatology	0.802 (0.774–0.829)	0.203	0.891	0.152
Persistence	0.750 (0.724–0.778)	0.428	0.508	0.789

CI: gauge-cluster bootstrap. LSTM simulation uses observed future rain.

where Q is observed streamflow and q_g^{95} the 95th percentile of the gauge's flow in the training years only. The 50 inputs (streamflow, rainfall, weather, soil moisture, season and catchment attributes) are all known at the end of day t. We train on 193 gauges for 1980–2008, select the model and its decision threshold on 2009–2014, and test once on 48 unseen gauges in 2015–2020, labelled by (1) (52,994 gauge-days, 5.5% flood windows). A depth-6 XGBoost model [24] won the validation comparison.

Results. Table II compares it with persistence, the gauge's own climatology and the CAMELS-IND regional LSTM simulation, which is driven by the observed future rain and so has an advantage no forecaster has. The model led on balanced accuracy (BA) and on PR-AUC (Fig. 2). Scored on random rows instead, the same configuration reached a BA of 0.915 and a PR-AUC of 0.810, against 0.898 and 0.736 on the held-out split (both from the 500,000-row refit used for the protocol comparison): the protocol alone moved PR-AUC by 7.4 points. Streamflow inputs carried 67.5% of the model's gain, and recall fell from 96.7% one day ahead to 89.5% three days ahead. Skill rose with catchment area (Spearman $\rho = 0.44$, $p = 0.002$), and 86% of false alarms came in the monsoon.

Check. A precision of 0.328 means that most alarms are false, so in the chain an alarm opens a review and never an order. The model without streamflow inputs is the fallback for a lost gauge feed: its BA was lower, 0.853, but it warned of 94.0% of the 266 flood onsets in advance, against 91.4% for the full model, because onsets depend on rain more than on the current river state. In the chain, the data-health flag that the policy agent reads (Section VII) would switch the forecast to this fallback; this switch is designed but not yet built. The raw

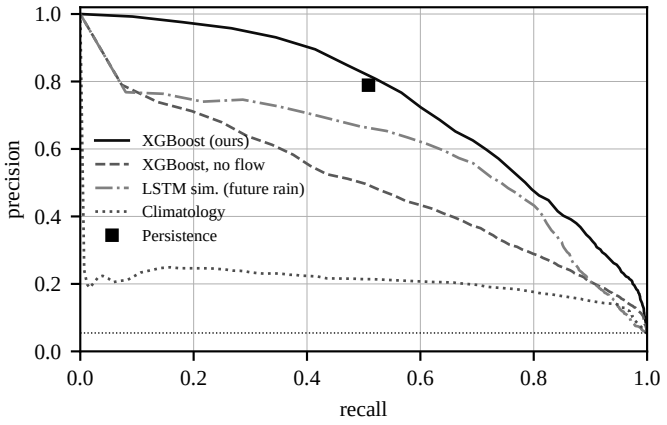


Fig. 2. Precision–recall curves on the unseen test gauges. The square is persistence; the dotted line is the flood rate.

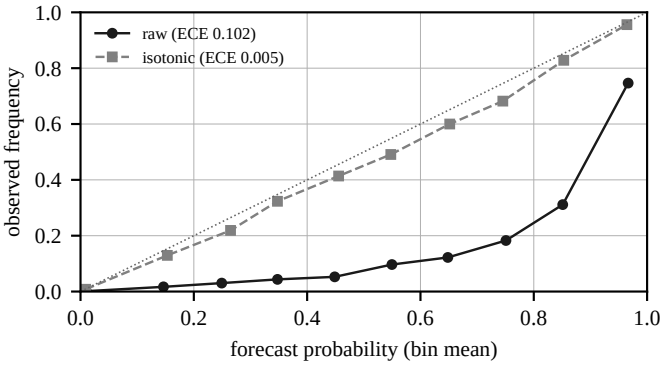


Fig. 3. Reliability of the forecast probabilities on the unseen test gauges, before and after isotonic recalibration fitted on validation data (bins with at least 50 gauge-days).

probabilities were over-confident (expected calibration error, ECE, 0.102): in the highest well-populated bin the forecast averaged 0.97 while floods followed in 0.75 of cases. Isotonic recalibration fitted on validation data lowered the ECE to 0.005 (Fig. 3), so the number shown to an officer can be read as a frequency.

V. Shelter Re-Planning with QAOA

Formulation. Groups g of size n_g are assigned to shelters s with R_s seats left. With travel cost c_{gs} on the best open path (flood- and congestion-weighted) and a set $\mathcal{F}$ of forbidden pairs (no open path, failed shelter, group larger than the seats left), a plan $\mathbf{x} \in \{0,1\}^{G\times S}$ is valid when

$$V(\mathbf{x}) = \Big[\textstyle\sum_s x_{gs} = 1 \ \forall g\Big] \wedge \Big[x_{gs} = 0 \ \forall (g,s) \in \mathcal{F}\Big] \wedge \Big[\textstyle\sum_g n_g x_{gs} \le R_s \ \forall s\Big]. \quad (2)$$

Capacity is an inequality that normally needs slack qubits. We encode it pairwise instead: with $\mathcal{O}$ the pairs of groups that together exceed R_s and shifted costs $c'_{gs} = c_{gs} - \min_{s'} c_{gs'}$,

$$E(\mathbf{x}) = \sum_{g,s} c'_{gs} x_{gs} + P\Big[\sum_g \Big(1 - \sum_s x_{gs}\Big)^2 + \sum_{(g,s)\in\mathcal{F}} x_{gs} + \sum_{(g,h,s)\in\mathcal{O}} x_{gs}x_{hs}\Big]. \quad (3)$$

After scaling every instance to the energy range $[0,1]$, all instances share one scale. The pairwise term cannot see an overload by three or more groups whose pairs all fit, so a minimum of (3) need not satisfy (2); Section VI measures how often this happens.

Mixers and transfer. We simulate QAOA [16] exactly on 12-qubit twin instances (four groups, three shelters) after a road closure and a shelter surge. The transverse-field (X) mixer explores all bitstrings; a ring-XY mixer [17], [18] swaps amplitude inside each group, so every state stays a one-hot plan; a Grover mixer [25] mixes all one-hot plans in one step but needs far deeper circuits. For transfer, we fix at each depth one trained angle set from 30 development instances, in the spirit of [19], and apply it unchanged to new instances. Choosing the set with the highest mean optimal-plan probability uses the exact optima of the development set, so it is an oracle; we also test rules that need no optima.

Results. Over 30 instances and 5 optimizer seeds, the XY mixer placed 0.188 ± 0.020 of its probability on the optimal plan at depth 4, against at most 0.048 for the X mixer, and beat it on at least 25 of 30 instances at every depth (Holm-adjusted $p \le 8.0 \times 10^{-6}$). The Grover mixer was not significantly different from XY ($p \ge 0.19$). Table III gives the transfer results. On 30 held-out twin instances the fixed angles beat per-instance training, at no per-instance cost. On 50 instances built on the road graphs of 6 cities they matched it ($p = 0.74$) and found the optimum within 32 shots more often (89.9% against 74.9%, Holm $p = 0.042$). Chosen on the twin by mean energy or CVaR, which need no optima, the set fell below per-instance training in leave-one-out (0.082 and 0.164 against 0.215). A third rule needs no optima: it scores each set by the expected cost of the best verified plan among 32 shots. On 195 fresh city instances under four disruption patterns, its set reached the optimum within 32 shots in 96.8% of cases, against 80.1% for per-instance training and 90.6% for CVaR training. Under three new disruptions no fixed set raised P_{opt} significantly, and 3 of nine comparisons lowered it.

Classical baselines. Exhaustive search over the 81 one-hot plans took 0.90 ms per instance. A greedy rule found a feasible plan in 100.0% of instances and the optimum in 93.3%, and simulated annealing with 20 restarts reached the optimum in 80.0% in 128 ms. QAOA is therefore neither the fastest nor the most reliable solver at this size.

VI. Runtime Assurance for the Re-Planner

Algorithm 1 puts it inside a runtime-assurance loop [11], [12]. Every sample is checked with (2) against the state at dispatch time. If none passes, a greedy rule (largest group first to its cheapest shelter with seats left) is tried. Only then

TABLE III
ANGLE TRANSFER AT DEPTH 4 (XY MIXER, 12 QUBITS)

Measure	Trained per instance	Fixed angles*
Held-out twin, P_{opt}	0.163	0.306
Held-out, optimum in 32 shots (%)	79.7	99.99
Held-out, circuit evaluations	2,496	0
Six cities, P_{opt}	0.213	0.202
Cities, optimum in 32 shots (%)	74.9	89.9
Fresh cities, no-optima rule (%)	80.1	96.8

*Other rows: set chosen with development optima (oracle).

Algorithm 1 Runtime-assurance re-planning at one event

```
draw shots in batches until a sample passes V or the budget is used
x ← cheapest sample with V(x); if none, x ← greedy
if V(x) then
    dispatch x
else
    solve the MILP with a time limit; if feasible, dispatch its plan
    if infeasible, escalate with the certificate; if timed out, escalate as uncertified
end if
```

does the loop ask a mixed-integer program (MILP), solved with HiGHS [26], to minimise $\sum c_{gs}x_{gs}$ subject to (2). An infeasible MILP certifies that no valid plan exists, so the escalation to a human is justified; a feasible one supplies the plan. A solver time-out proves nothing, so that case is escalated but marked uncertified. The natural baseline is the uncertified loop, which escalates whenever the sampler and the greedy rule fail.

Setup. On the twin we replay 30 seeded episodes of closures, a shelter surge and two shelter failures (180 events, 27 without any valid plan), each sampled ten times by depth-3 XY QAOA and by simulated annealing [27] on the same QUBO. At city scale we built 360 instances on the road graphs of the six cities with 8, 16 or 32 groups and ten shelters (80–320 variables), where enumeration is impossible and the MILP is the only exact reference.

Results. Table IV gives the outcome. On the twin, dispatching the most frequent sample unchecked would have sent an invalid plan in 29.1% of event draws for QAOA and 20.6% for annealing (16.6% and 6.5% on solvable events only). Almost all of these samples overfilled a shelter in exactly the way the pairwise penalty cannot see, so the error lies in the encoding, not in the search. At city scale the search failed instead: the annealer's rate of valid plans on solvable instances fell from 71.3% at 8 groups to 17.0% at 32 (Fig. 4), and the uncertified loop would have called a human for 4.4% of cases that had a plan. The MILP settled all 360 instances, the slowest in 40.8 s, but a 1 s deadline would have stopped 45 of them, all with valid plans. Calling the MILP first with that deadline, and keeping the sampler and greedy rule as fallback, dispatched an optimal plan in 82.5% of solvable calls, against 37.0% when the sampler went first. An adaptive shot budget cut the twin's shots per event from 64 to 18.4 with identical escalation

TABLE IV
RE-PLANNING WITH AND WITHOUT THE CHECKS

Measure	Value
Twin: 180 events × 10 draws, 12 variables	
Unchecked top sample invalid, QAOA (%)	29.1 (22.7–35.6)
Unchecked top sample invalid, annealing (%)	20.6 (14.3–26.4)
Invalid samples the penalty could not see, QAOA (%)	97.7
Invalid plans dispatched by the loop	0
Cities: 360 instances, 80–320 variables	
Annealing finds a valid plan, 8 / 32 groups (%)	71.3 / 17.0
Solvable cases escalated, uncertified loop (%)	4.4 (2.3–6.9)
Settled by MILP within 60 s	360 of 360
Optimal plan, solver first, 1 s deadline (%)	82.5

Brackets: 95% intervals from resampling events or instances.

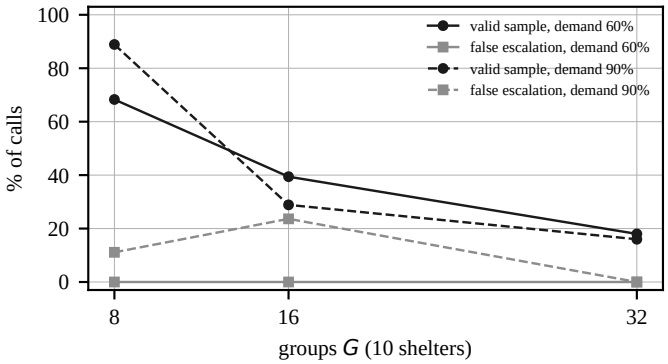


Fig. 4. City scale: share of calls in which the sampler produced a valid plan, and share of solvable cases an uncertified loop would escalate, against the number of groups.

decisions, but optimal QAOA plans fell from 93.1% to 68.8% of events.

Differences between cities. Unverified dispatch was unsafe in 12.7 to 100.0% of calls, depending on the city. In Kolkata the three closures cut one of the six zones off from every shelter, so any instance with a group there has no valid plan, and only 5 of 60 were solvable. Among the other five cities, Chennai had the most false escalations (8.0%) and the weakest greedy rule, which found a plan in 92.0% of its solvable instances. The failures of the uncertified loop therefore depend on the city network as well as on the problem size. The MILP's median solve time grew from 0.01 s at 8 groups to 0.32 s at 32, so a typical certificate takes under a second, though the slowest took 40.8 s.

VII. AGENTS AND LLM NARRATION

Policy and gate. The Flood Risk agent maps a flood probability p and three health flags (data available, model returned a value, database connected) to one of five actions. With a monitor threshold $\tau_m = 0.50$ and an automation threshold $\tau_a = 0.60$, the policy π returns MONITOR below τ_m, AUTO only when $p \geq \tau_a$, all flags are healthy and automation is enabled, REVIEW otherwise, and OVERRIDE or UNAVAILABLE when an operator overrides or no probability exists. A Random Forest [28] and an XGBoost classifier score the same features;

dispatch is held when they differ by at least 0.15 or fall on opposite sides of τ_a (the straddle rule).

Composed check. An exhaustive test of π over 16,032 cases (1,002 probability values including an undefined one, eight health combinations and two override states) found 0 violations. The path from policy to dispatch was a different matter. Enumerating 32,032 cases through the dispatch code (1,001 defined probabilities, eight health combinations, two override states and the two states of the hold) showed that teams were sent in 2,807 cases in which the verified policy asked for review, all of them high-probability states with a failed health check: dispatch tested the probability band and the gate but never read the action returned by π. After the fix (dispatch requires AUTO or OVERRIDE) the same check found 0. On 910 held-out rows of the classifiers' data, cases held by the gate were wrong 1.8 times as often as released ones; the gate caught 40.3% of the policy model's errors. The straddle rule alone held fewer cases (7.6% against 26.9%) with a higher error rate among them, but it caught only 15.9% of the errors, so it is not a better gate.

Numeric guard. The LLM receives the logged state and must use only its numbers; it has no actuation rights. The deployed guard accepts a reply only if every number in it matches some number of the prompt up to the rounding written, and replaces a rejected reply by a template filled from the same state. Because it is untyped, a rainfall value quoted as a river level passes. A slot-typed guard first binds each number to a quantity (percent to a probability, mm to rainfall, m to a level, clock times to the current time) and rejects any number it cannot bind.

Live replies. We collected 300 replies from gpt-oss-20b and gpt-oss-120b on 150 seeded flood states. 109 of them (36.3%) stated at least one unsupported number, and 106 of those were invented arrival times, although the prompt contains none. A revised typed guard was frozen, with its hash recorded, before replies from a third model family, ALLaM-2 (7B), were collected. Table V gives the results. The typed guard flagged more of the labelled errors than the deployed one but rejected more correct replies. An LLM judge [22] flagged nearly all labelled errors; it rejected 14.1% of correct replies when its 23 failed calls count as rejections and 6.8% without them, and it needs a model call per message. On the held-out family the revised guard was not significantly better than the deployed one (McNemar $p = 0.62$), and its false rejections rose from 0.0% to 12.5%. No numeric guard sees qualitative over-claims such as "flood confirmed", made in 14.3% of replies. The kind of error also changed with the model (Fig. 5): invented arrival times appeared in 32–39% of the gpt-oss replies but in only 4.5% of the ALLaM-2 replies, which instead invented quantities in 39% and misquoted a probability in 16%. A guard tuned to the errors of one model family will therefore miss some errors of the next one, and a typed check of every quantity is safer than a list of known mistakes.

Classifier calibration. The disagreement gate held many cases because the two classifiers were calibrated differently: on the held-out rows the XGBoost policy model had an ECE of 0.108, against 0.044 for the Random Forest. Isotonic calibration on the training split lowered these to 0.018 and 0.031 and the hold rate of the deployed rule to 10.5%. The forecast layer and the gate thus need the same repair before their probabilities are compared with a fixed threshold.

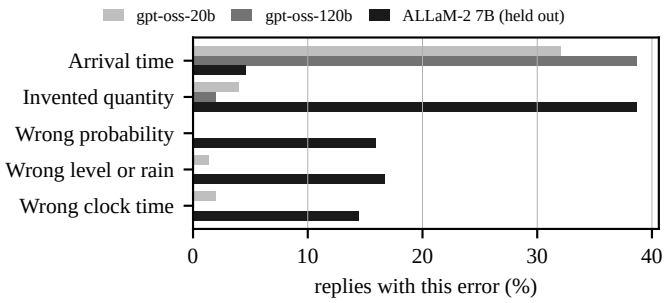


Fig. 5. Kinds of unsupported numbers in live replies, as a share of each model's replies (reference labels). A reply can contain more than one kind.

TABLE V
GUARDS ON LIVE LLM REPLIES: RECALL ON REPLIES WITH AN UNSUPPORTED NUMBER AND FALSE REJECTION (FR) OF CORRECT REPLIES, IN %

	gpt-oss (300)		ALLaM-2, held out (132)	
Guard	Recall	FR	Recall	FR
Deployed (untyped)	89.0	5.8	96.1	7.1
Slot-typed	93.6	10.5	100.0	17.9
Slot-typed v2 (frozen)	100.0	0.0	98.7	12.5
LLM judge	99.1	14.1	–	–

v2 was written after reading the gpt-oss replies; – = not run.

VIII. DISCUSSION

Check the composed path, not only the component. The verified policy had no violations, yet the dispatch code bypassed it in 2,807 cases. The forecaster's score depended on the split in the same way: random rows added 7.4 points of PR-AUC that a new river would never deliver. In both cases the component was correct and the path around it was not.

Check proposals against typed state. The two generative components failed for parallel reasons. The pairwise penalty could not see a three-group overload, and the untyped guard could not see a correct number placed in the wrong slot. Both checks compared the output with the right data at the wrong granularity. An exact verifier of (2) and a typed guard close these gaps, because each checks every constraint or quantity by name.

A check that cannot decide must say so. A MILP time-out is not a proof of infeasibility: every city instance that a 1 s deadline would have stopped (45) had a valid plan. The loop therefore escalates such cases marked as uncertified, and the forecaster's alarms go to review because its precision is 0.328.

Every check has a price. The typed guard rejected 10.5% of correct replies against 5.8% for the deployed one; the adaptive shot budget lost optimal plans; the no-flow fallback lost BA. We report these costs so that an operator can choose the operating point.

Reproducibility. All experiments use fixed seeds, and the live LLM replies are stored. A script writes every number into the source and asserts each comparison the text makes, so a changed result fails the build.

Threats to validity. The forecast benchmark uses peninsular gauges of CAMELS-IND, while the agents and the twin model the Delhi Yamuna corridor, and the policy's classifiers were trained on separate data; joining the forecaster to the policy is future work. Twin and city instances use synthetic group sizes and estimated shelter capacities. All QAOA runs are classical simulations (12 qubits here, at most 27 in the companion study), and exact methods solve such instances faster. The LLM results cover three model families and one prompt. No field trial with officers has been made.

IX. Conclusion

A flood decision-support chain can use a learned forecaster, a sampling optimizer and an LLM narrator safely only if none of them acts on its own output. In our prototype each one proposes and a deterministic check decides. Measured on held-out data and on the deployed code, the checks found a policy bypass in 2,807 cases, blocked every invalid shelter plan, separated certified from uncertified escalations, and flagged 93.6% of the replies with unsupported numbers in live LLM output, each at a cost that we report. A better component, such as a constraint-preserving QAOA mixer with fixed angles or a stronger forecaster, raises the quality of the proposals but does not remove the need to check them. We make no claim of quantum advantage.

Acknowledgment

The authors thank the CAMELS-IND authors, the OpenStreetMap contributors and Open-Meteo for openly available data. In line with the IEEE policy on AI-generated content, the authors disclose that Claude (Anthropic) was used to help edit and restructure the text of all sections and to draft the simulation, experiment and plotting scripts. All experiments use the authors' own software and fixed seeds; the live LLM replies are stored with the scripts. The authors checked every number, claim and reference and take full responsibility for the content.

References

[1] F. Kratzert, D. Klotz, G. Shalev, G. Klambauer, S. Hochreiter, and G. Nearing, "Towards learning universal, regional, and local hydrological behaviors via machine learning applied to large-sample datasets," *Hydrol. Earth Syst. Sci.*, vol. 23, pp. 5089–5110, 2019, doi: 10.5194/hess-23-5089-2019.

[2] G. Nearing *et al.*, "Global prediction of extreme floods in ungauged watersheds," *Nature*, vol. 627, pp. 559–563, 2024, doi: 10.1038/s41586-024-07145-1.

[3] A. Lucas, "Ising formulations of many NP problems," *Front. Phys.*, vol. 2, Art. no. 5, 2014, doi: 10.3389/fphy.2014.00005.

[4] A. Abbas *et al.*, "Challenges and opportunities in quantum optimization," *Nat. Rev. Phys.*, vol. 6, pp. 718–735, 2024, doi: 10.1038/s42254-024-00770-9.

[5] A. R. Cadeliña *et al.*, "D-Wave implementation of quantum annealing for optimal resource allocation in disaster response operation of Marikina City," *ECTI Trans. Comput. Inf. Technol.*, vol. 18, no. 1, pp. 24–33, 2024, doi: 10.37936/ecti-cit.2024181.253577.

[6] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[7] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, Art. no. 100804, 2023, doi: 10.1016/j.patter.2023.100804.

[8] D. R. Roberts *et al.*, "Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure," *Ecography*, vol. 40, no. 8, pp. 913–929, 2017, doi: 10.1111/ecog.02881.

[9] F. Glover, G. Kochenberger, and Y. Du, "Quantum bridge analytics I: A tutorial on formulating and using QUBO models," *4OR*, vol. 17, no. 4, pp. 335–371, 2019, doi: 10.1007/s10288-019-00424-y.

[10] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, Art. no. 248, 2023, doi: 10.1145/3571730.

[11] L. Sha, "Using simplicity to control complexity," *IEEE Softw.*, vol. 18, no. 4, pp. 20–28, 2001, doi: 10.1109/MS.2001.936213.

[12] A. Desai, S. Ghosh, S. A. Seshia, N. Shankar, and A. Tiwari, "SOTER: A runtime assurance framework for programming safe robotics systems," in *Proc. 49th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*, 2019, pp. 138–150, doi: 10.1109/DSN.2019.00027.

[13] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1, 2018, pp. 2669–2678, doi: 10.1609/aaai.v32i1.11797.

[14] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern. A, Syst. Humans*, vol. 30, no. 3, pp. 286–297, 2000, doi: 10.1109/3468.844354.

[15] N. K. Mangukiya *et al.*, "CAMELS-IND: Hydrometeorological time series and catchment attributes for 228 catchments in Peninsular India," *Earth Syst. Sci. Data*, vol. 17, pp. 461–491, 2025, doi: 10.5194/essd-17-461-2025.

[16] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," arXiv:1411.4028, 2014.

[17] S. Hadfield, Z. Wang, B. O'Gorman, E. G. Rieffel, D. Venturelli, and R. Biswas, "From the quantum approximate optimization algorithm to a quantum alternating operator ansatz," *Algorithms*, vol. 12, no. 2, Art. no. 34, 2019, doi: 10.3390/a12020034.

[18] Z. Wang, N. C. Rubin, J. M. Dominy, and E. G. Rieffel, "XY mixers: Analytical and numerical results for the quantum alternating operator ansatz," *Phys. Rev. A*, vol. 101, no. 1, Art. no. 012320, 2020, doi: 10.1103/PhysRevA.101.012320.

[19] R. Shaydulin, P. C. Lotshaw, J. Larson, J. Ostrowski, and T. S. Humble, "Parameter transfer for quantum approximate optimization of weighted MaxCut," *ACM Trans. Quantum Comput.*, vol. 4, no. 3, Art. no. 19, 2023, doi: 10.1145/3584706.

[20] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *Proc. Conf. Empir. Methods Nat. Lang. Process.: Syst. Demonstrations*, 2023, pp. 431–445, doi: 10.18653/v1/2023.emnlp-demo.40.

[21] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM 48th Int. Conf. Softw. Eng. (ICSE)*, 2026, pp. 2938–2950, doi: 10.1145/3744916.3764546.

[22] L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Adv. Neural Inf. Process. Syst. 36 (Datasets Benchmarks Track)*, 2023.

[23] N. G. Leveson, *Engineering a Safer World: Systems Thinking Applied to Safety*. Cambridge, MA, USA: MIT Press, 2011.

[24] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[25] A. Bärtschi and S. Eidenbenz, "Grover mixers for QAOA: Shifting complexity from mixer design to state preparation," in *Proc. IEEE Int. Conf. Quantum Comput. Eng. (QCE)*, 2020, pp. 72–82, doi: 10.1109/QCE49297.2020.00020.

[26] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Math. Program. Comput.*, vol. 10, no. 1, pp. 119–142, 2018, doi: 10.1007/s12532-017-0130-5.

[27] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983, doi: 10.1126/science.220.4598.671.

[28] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.